NYU 上海 SHANGHAI 纽约大学

# Introduction to Computer Programming
# Fall 2026
# Mock Midterm

| | |
|---|---|
| **First Name** | |
| **Last Name** | |
| **NetID** | |

## Instructions

- **Duration**: 90 minutes
- **Max score**: 100 pts

- Computers, calculators, phones, textbooks or notebooks are **not allowed**
- When writing programs, make very clear your line **indentations** (there are some indentation levels in the areas where you will write code). If indentation is not perfectly clear, the grader will consider "no indent".
- We can **not answer** questions during the test.
- If you are unsure of a problem, please write down your assumptions, and proceed with it. We will grade accordingly.
- **Do not** use lists, string methods (such as .split() or .count()), dictionary, set or define functions. Use only concepts covered in class.

## Grading

| **Questions** | **Q1** | **Q2** | **Q3** | **Q4** | **Q5** | **Q6** | **Q7** | **Q8** |
|---|---|---|---|---|---|---|---|---|
| **Points** | $8'$ | $8'$ | $24'$ | $10'$ | $10'$ | $10'$ | $15'$ | $15'$ |
| **Score** | | | | | | | | |
| **Total** | | | | | | | | |

## Question 1 | Boolean Expression

Evaluate the following boolean expression:

| | |
|---|---|
| `not True or True` | |
| `3 != '3' or 3 > 5` | |
| `1 < ('Bee' == 'bee') and 5//2  < 1` | |
| `not False or False and True` | |

## Question 2 | Binary and Hexadecimal Representaion

Complete the table below by converting each given number into the other two number systems: decimal, binary, and hexadecimal.

| Decimal | Binary | Hexadecimal |
|---|---|---|
| 45 | | |
| | | 3F |

## Question 3 | Predict the Output

**Program 1**

```
print(23 // 4)
print(23 % 4)
```

**Output 1**

**Program 2**

```
a = 5
b = 2

print(a, b, sep=" + ", end=" = ")
print(a + b)
print(f"{a / b:.2f}")
```

**Output 2**

**Program 3**

```
word = "PROGRAM"

print(word[1:5])
print(word[-2:2])
print(word[-3:])
print(word[::-2])
print(word[-1:3:-1])
```

**Output 3**

| Program 4 | Output 4 |
|---|---|
| `for i in range(1, 8):`<br>`  if i % 3 == 0:`<br>`  continue`<br>`  if i > 5:`<br>`  break`<br>`  print(i)`<br><br>`print("End")` | |
| **Program 5** | **Output 5** |
| `n = 19`<br><br>`while n > 0:`<br>`  print(n)`<br>`  n //= 2` | |
| **Program 6** | **Output 6** |
| `result = ""`<br><br>`for ch in "a1b2c3":`<br>`  if ch in "123":`<br>`  result = ch + result`<br><br>`print(result)` | |
| **Program 7** | **Output 7** |
| `x = 12`<br><br>`if x % 3 == 0:`<br>`  print("A")`<br>`  if x % 5 == 0:`<br>`  print("B")`<br>`  else:`<br>`  print("C")`<br>`else:`<br>`  print("D")` | |
| **Program 8** | **Output 8** |
| `total = 0`<br><br>`for i in range(2, 8, 2):`<br>`  total += i`<br>`  print(i, total)` | |

## Question 4 | Predict the Output

The string seats represents a row of cinema seats. X indicates an occupied seat, and `.` indicates an available seat. Write the exact output of the following program.

```
seats = "X..X...X...."

for group_size in range(2, 6):
    position = 0
    available = 0

    while position < len(seats):
        if seats[position] == ".":
            available += 1
        else:
            available = 0

        if available == group_size:
            first_seat = position - group_size + 2
            print(f"Group {group_size}: seats "+
                  f"{first_seat}-{position + 1}")
            break

        position += 1

    if available < group_size:
        print(f"Group {group_size}: no suitable seats")
```

**Your answer :**

## Question 5 | Triangle Classifier

Write a Python program that reads three positive integers representing the side lengths of a triangle.

Classify the input as follows:

- **Invalid triangle:** The sum of any two sides is less than or equal to the third side.
- **Equilateral triangle:** All three sides are equal.
- **Isosceles triangle:** Exactly two sides are equal.
- **Scalene triangle:** All three sides are different.

**Example Output** (user input is in bold)**:**

**Example 1:**

*Enter side 1:* ***5***
*Enter side 2:* ***5***
*Enter side 3:* ***3***

*Isosceles triangle*

**Example 2:**

*Enter side 1:* ***2***
*Enter side 2:* ***3***
*Enter side 3:* ***5***

*Invalid triangle*

**Your code:**

## Question 6 | Digit Counter

Write a Python program that reads a positive integer `n` and a threshold `k`, then displays how many digits of `n` are **greater than or equal to** `k`.

**Example Output** (user input is in bold)**:**

*Enter a positive integer:* ***431663334231***
*Enter a threshold (0-9):* ***3***
*Count: 9*

**Your code:**

## Question 7 | Message Compressor

A simple message compressor replaces each consecutive group of identical characters with the character followed by the number of times it appears. Write a program that reads a message and prints its compressed form.

**Example Output** (user input is in bold)**:**

**Example 1:**

*Enter a message:* ***aaabbcaaaa***
*Compressed: a3b2c1a4*

**Example 2:**

*Enter a message:* ***AAAaa***
*Compressed: A3a2*

**Your code:**

## Question 8 | Dice Duel

Two players, Alice and Bob, play a dice game. In each round, the computer rolls one six-sided die for each player.

- If Alice's roll is higher, Alice earns one point.
- If Bob's roll is higher, Bob earns one point.
- If the rolls are equal, neither player earns a point. Print `Tie! Roll again.` and roll again for both players in the same round.
- The first player to earn **3** points wins.

Write a Python program that simulates the game and follows the output format shown in the example below.

**Example Output:**

```
Round 1
Alice rolls 4, Bob rolls 4
Tie! Roll again.
Alice rolls 6, Bob rolls 2
Score: Alice 1, Bob 0

Round 2
Alice rolls 1, Bob rolls 5
Score: Alice 1, Bob 1

Round 3
Alice rolls 3, Bob rolls 2
Score: Alice 2, Bob 1

Round 4
Alice rolls 2, Bob rolls 2
Tie! Roll again.
Alice rolls 5, Bob rolls 3
Score: Alice 3, Bob 1

Alice wins!
Rounds played: 4
```

**Your code:**

*You may tear off this cheat sheet page for reference during the exam.*

# ICP Cheat Sheet

**Binary, Hexadecimal**

```
bin(10)     0b1010
hex(90)     0x5A
```

**Data type conversion**

```
float('2')   2.0   | float(2)     2.0
int('1')     1     | int(1.9)     1
str(12)      '12'  | str(12.1)    '12.1'
```

**Scientific notation**

```
10e2  #100    0e-2   #0.1
```

**Escape sequence**

| | | | |
|---|---|---|---|
| \n | new line | \\ | back slash |
| \t | tab (8 characters) | \' | single quote |

**Data type check**

```
type(12)     int   | type('ICP')   str
type(1.0)    float | type(True)    bool
                   | type(False)   bool
```

**Comparison**

```
1 == 1     True    | 1 != 1     False
1 < 2      True    | 2 <= 2     True
1 > 2      False   | 2 >= 2     True
1=='1'     False   | 1 != '1'   Ture
```

**Membership**

```
'a' in 'abc'       #True
'a' not in 'abc'   #False
'a' in 'ICP'       #False
'a' not in 'ICP'   #True
```

**Boolean operators**

```
not True       #False   not False #True
True and True  #True
True and False #False
False and False #False
True or True #True   True or False #True
False or False #False
```

**Division**

```
5/2       # 2.5
```

division operation always returns a float

**Integer division**

```
5//2      # 2          -5//2     #-3
6//4.0    # 1.0
```

**Modulo**

```
5%2       # 1          -5%2      # 1
10%4.0    # 2
```

**Operator precedence (For highest to lowest)**

| | |
|---|---|
| () | parentheses |
| ** | exponentiation |
| +, - | unary + and - (signs) |
| *, /, //, % | multiplication, division integer division and modulo |
| +,- | addition and subtraction |
| ==, !=, <, <=, >=, >, in, not in | comparison operators/membership |
| not<br>and<br>or | Logic operators |

**Print function**

```
print('a','b',sep='*')           # a*b
print('c','d',sep='?',end='*')   # c?d*e f
print('e','f')
```

**Input function**

```
name = input("Enter your name: ")
print("Your name is: " + name)
#Enter your name: ICP
#Your name is: ICP
```

**range function**

```
range(2,10,2)        range(10,0,-2)
#2,4,6,8             #10,8,6,4,2
range(1,4)           range(4,1)  #empty
#1,2,3               sequence
range(4)             range(-4)
#0,1,2,3             #empty sequence
```

**Module**

```
import math, random
math.sqrt(4)   #2       math.ceil(1.2) #2
math.floor(1.2)#1

random.random()
#a random float between 0 and 1(included)
random.randint(1,10)
#a random int between 1 and 10(included)
```

**Decision Structure**

```
if n == 0:
   print("n is zero")
elif n > 0:
   print("n is strictly positive")
else: # n < 0
   print("n is strictly negative")
```

**Repetition Structure**

```
n=0                          0
while n < 4:                 1
  print(n)                   2
  n+=1                       3
```

```
num=input('A number(0 to quit): '))
while num != 0:
    num=input('A number(0 to quit): '))
```

```
for i in range(3):           0
    print(i)                 1
                             2
```

```
for i in 'abc':              a
    print(i)                 b
                             c
```

**String Delimiters**

Single quotes: `'I am a string'`
Double quotes: `"I'm a string"`
Triple single quotes: `'''I'm a string'''`
Triple double quotes: `"""I'm a string"""`

**f-string**

```
print(f'Cost is {12.3456:.2f}$!')
#Cost is 12.35$!
```

**String Operations**

Concatenation : `'ABC'+'3'` `#'ABC3'`
Repetition: `'ABC'*3` `#'ABCABCABC'`

**String Length**

```
len('0123456789') # 10
len('\n')  # 1    len('') # 0
```

**Unicode Code/Text Conversion**

```
>>> ord('a') #97  >>> chr(97)  # 'a'
>>> ord('A') #65  >>> chr(65)  # 'A'
```

**String Indexing `S='ABCDEF123456'`**

```
len(S) # 12
S[0]   # 'A'        S[-1]  # '6'
S[11]  # '6'        S[-12] # 'A'
```

**String Slicing `S='ABCD1234'`**

```
S[:4]    #'ABCD'   | S[-4:]    #'1234'
S[4:]    #'1234'   | S[-6:-2]  #'CD12'
S[2:6]   #'CD12'   | S[-1:-8]  #''
S[4:1]   # ''      | S[:5:2]   #'AC1'
S[::-1]  #'4321DCBA'
S[1:7:2]#'BD2'     S[7:0:-3]#'41B'
```